action: `created` indicates that we successfully indexed a new document in Elasticsearch's store. The `_index`, `_type`, and `_id` attributes are derived from our incoming request and assigned to the document. `_version` indicates the current version of this document; the value `1` means the first version of the document. The number is incremented if we modify the document and reindex it. We visit the `_shards` information down the line, but all it's saying here is that the document's journey to the store was completed.

If you are curious, re-execute the same command and check the response. You'll notice that the `result` attribute changes to `updated` and the version number increments. Every time you execute the query, the version is updated.

When we first tried to insert the document, no `movies` index had been pre-created for us (if the index already exists, be sure you delete it by running the `DELETE movies` command). When the document was first seen by the server, the server realized the index didn't exist yet, and it created a `movies` index for us so our document indexing went smoothly. The schema of this index was also nonexistent, but the server derived the schema by analyzing the first incoming document. We learned about mapping definitions in chapter 4; refer to section 4.1.1 if needed.

We just indexed a document that has an ID. Not every document can or will have an identity with a business need. For example, while it's a good idea for a `movie` document to have an ID, traffic alerts from traffic lights received by a traffic management system don't need to be associated with an ID. Similarly, tweet streams, cricket match commentary messages, and so on don't need an identity—a random ID will suffice.

Indexing a document without an ID follows the same process as indexing one with an ID, except the HTTP verb changes from `PUT` to `POST`. Let's see this in action.

#### INDEXING A DOCUMENT WITHOUT AN IDENTIFIER (POST)

In the previous section, we indexed a document that had an ID associated with it. We used the HTTP `PUT` method to create the resource in Elasticsearch. But not every business model promotes data documents with IDs (for example, stock price ticks or tweets). In such cases, we can let Elasticsearch generate random IDs. To index these documents, instead of using the `PUT` method as we did earlier, we use the `POST` action.

The HTTP `POST` method follows a format similar to `PUT`, but the ID is not provided as part of the URL parameter.

**Listing 5.2 Indexing a document with no ID using `POST`**

```
POST myindex/_doc
{
  "title":"Elasticsearch in Action"
}
```

The URL has no ID attached to it.

The request body is a JSON document.

We invoke the `_doc` endpoint on the index *without an ID* but with a body. This `POST` request tells Elasticsearch that it needs to assign the documents newly generated random IDs during the indexing process.